



北京師範大學  
BEIJING NORMAL UNIVERSITY

## 计算机图形学 实验报告

姓名 杨博文 学号 202311081015  
院系 人工智能学院 专业 计算机科学与技术

2025 年 3 月 28 日

### 摘 要

计算机图形学的第三次实验，主要内容包括为 LookAt 矩阵，即摄像机位置进行调整。

# 目录

|          |                           |           |
|----------|---------------------------|-----------|
| <b>1</b> | <b>运行环境及运行方式</b>          | <b>3</b>  |
| <b>2</b> | <b>任务 1: 将摄像机放置在三个位置上</b> | <b>3</b>  |
| 2.1      | 任务要求 . . . . .            | 3         |
| 2.2      | 关键代码 . . . . .            | 3         |
| 2.3      | 结果图片 . . . . .            | 4         |
| <b>3</b> | <b>任务 2: 键鼠控制摄像机</b>      | <b>5</b>  |
| 3.1      | 任务要求 . . . . .            | 5         |
| 3.2      | 关键代码 . . . . .            | 5         |
| 3.3      | 结果图片 . . . . .            | 8         |
| <b>4</b> | <b>任务 3: 加入文字</b>         | <b>10</b> |
| 4.1      | 任务要求 . . . . .            | 10        |
| 4.2      | 结果 . . . . .              | 10        |

## 1 运行环境及运行方式

硬件为 Macbook Air, M2 芯片, 操作系统为 MacOS Sequoia 15.0。

IDE 为 Xcode Version16.0。

使用 GLFW+GLAD, 程序在 Xcode 内编译运行。

## 2 任务 1: 将摄像机放置在三个位置上

### 2.1 任务要求

在实验二中任务 3 的基础上, 对于设置深度测试的自定义立方体, 设置不同的相机位置、方向、坐标系 (至少 3 个不同的相机) 观察物体;

### 2.2 关键代码

```
glm::vec3 cameraPos(0.0f, 0.0f, -3.0f); // 摄像机位置
glm::vec3 cameraUp(0.0f, 1.0f, 0.0f); // 上向量
...
    //view
    glm::mat4 view = glm::lookAt(cameraPos, glm::vec3(0.0f, 0.0f, 0.0f), cameraUp);
    unsigned int viewLoc = glGetUniformLocation(shaderProgram, "view");
    glUniformMatrix4fv(viewLoc, 1, GL_FALSE, glm::value_ptr(view));
...
void processInput(GLFWwindow *window)
{
    if (glfwGetKey(window, GLFW_KEY_ESCAPE) == GLFW_PRESS)
        glfwSetWindowShouldClose(window, true);

    if (glfwGetKey(window, GLFW_KEY_1) == GLFW_PRESS)
    {
        cameraPos = glm::vec3(0.0f, 0.0f, -3.0f);
        cameraUp = glm::vec3(0.0f, 1.0f, 0.0f);
    }
    if (glfwGetKey(window, GLFW_KEY_2) == GLFW_PRESS)
    {
        cameraPos = glm::vec3(3.0f, 0.0f, 0.0f);
        cameraUp = glm::vec3(0.0f, 1.0f, 0.0f);
    }
}
```

```
if (glfwGetKey(window, GLFW_KEY_3) == GLFW_PRESS)
{
    cameraPos = glm::vec3(0.0f, 3.0f, 0.0f);
    cameraUp = glm::vec3(0.0f, 0.0f, -1.0f);
}
}
```

这样我们添加 view 矩阵，当键盘输入为 1、2、3 时调整该矩阵，得到不同的摄影机方向。我们固定看向原点。

## 2.3 结果图片

以下是我们的结果三视图，很显然的虽然感觉有点奇怪，但可以看出是一个完整的 3D 图形了。（这里由于我们在上周的任务中已经开启了深度，所以图层上没有遮挡问题）。

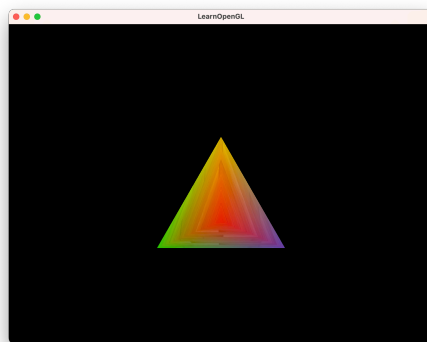


图 1: 实验 1，键盘输入 1 结果

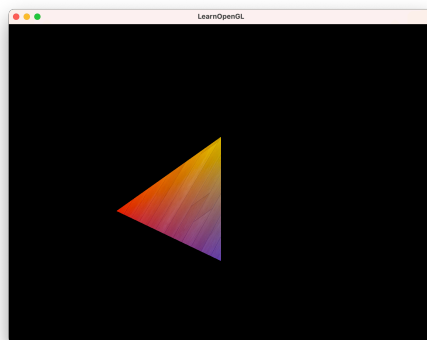


图 2: 实验 1，键盘输入 2 结果

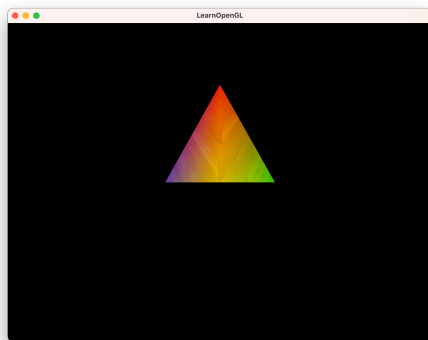


图 3: 实验 1, 键盘输入 3 结果

## 3 任务 2: 键鼠控制摄像机

### 3.1 任务要求

在任务 1 的基础上, 使用键盘与鼠标控制摄像机参数, 其中 WSAD 分别控制摄像机前后与左右平移, 鼠标控制摄像机方向;

### 3.2 关键代码

```
bool firstMouse = true;
float yaw = -90.0f; // yaw is initialized to -90.0 degrees since a yaw of 0.0 re
float pitch = 0.0f;
float lastX = 800.0f / 2.0;
float lastY = 600.0 / 2.0;
float fov = 45.0f;

// timing
float deltaTime = 0.0f; // time between current frame and last frame
float lastFrame = 0.0f;
...
glfwMakeContextCurrent(window);
glfwSetFramebufferSizeCallback(window, framebuffer_size_callback);
glfwSetCursorPosCallback(window, mouse_callback);
glfwSetScrollCallback(window, scroll_callback);

// tell GLFW to capture our mouse
glfwSetInputMode(window, GLFW_CURSOR, GLFW_CURSOR_DISABLED);
...
```

```
// per-frame time logic
// -----
float currentFrame = static_cast<float>(glfwGetTime());
deltaTime = currentFrame - lastFrame;
lastFrame = currentFrame;

//view
glm::mat4 view = glm::lookAt(cameraPos,
cameraPos+cameraFront,cameraUp);
unsigned int viewLoc = glGetUniformLocation
(shaderProgram, "view");
glUniformMatrix4fv(viewLoc, 1, GL_FALSE, glm::value_ptr(view));

...
void processInput(GLFWwindow *window)
{
    if (glfwGetKey(window, GLFW_KEY_ESCAPE) == GLFW_PRESS)
        glfwSetWindowShouldClose(window, true);

    if (glfwGetKey(window, GLFW_KEY_1) == GLFW_PRESS)
    {
        cameraPos = glm::vec3(0.0f, 0.0f, 3.0f);
        cameraFront = glm::vec3(0.0f, 0.0f, -1.0f);
        cameraUp = glm::vec3(0.0f, 1.0f, 0.0f);
    }

    float cameraSpeed = static_cast<float>(2.5 * deltaTime);
    if (glfwGetKey(window, GLFW_KEY_W) == GLFW_PRESS)
        cameraPos += cameraSpeed * cameraFront;
    if (glfwGetKey(window, GLFW_KEY_S) == GLFW_PRESS)
        cameraPos -= cameraSpeed * cameraFront;
    if (glfwGetKey(window, GLFW_KEY_A) == GLFW_PRESS)
        cameraPos -= glm::normalize(
            glm::cross(cameraFront, cameraUp)) * cameraSpeed;
    if (glfwGetKey(window, GLFW_KEY_D) == GLFW_PRESS)
        cameraPos += glm::normalize(
            glm::cross(cameraFront, cameraUp)) * cameraSpeed;
```

```
}  
void mouse_callback(GLFWwindow* window, double xposIn, double yposIn)  
{  
    float xpos = static_cast<float>(xposIn);  
    float ypos = static_cast<float>(yposIn);  
  
    if (firstMouse)  
    {  
        lastX = xpos;  
        lastY = ypos;  
        firstMouse = false;  
    }  
  
    float xoffset = xpos - lastX;  
    float yoffset = lastY - ypos;  
    lastX = xpos;  
    lastY = ypos;  
  
    float sensitivity = 0.1f;  
    xoffset *= sensitivity;  
    yoffset *= sensitivity;  
  
    yaw -= xoffset;  
    pitch -= yoffset;  
  
    if (pitch > 89.0f)  
        pitch = 89.0f;  
    if (pitch < -89.0f)  
        pitch = -89.0f;  
  
    glm::vec3 front;  
    front.x = cos(glm::radians(yaw)) * cos(glm::radians(pitch));  
    front.y = sin(glm::radians(pitch));  
    front.z = sin(glm::radians(yaw)) * cos(glm::radians(pitch));  
    cameraFront = glm::normalize(front);  
}  
void scroll_callback(GLFWwindow* window, double xoffset, double yoffset)
```

```
{  
    fov -= (float)yoffset;  
    if (fov < 1.0f)  
        fov = 1.0f;  
    if (fov > 45.0f)  
        fov = 45.0f;  
}
```

这里我们添加了对于键盘输入、鼠标输入的处理，统一了速度，进行了位置调整。键盘输入部分我们加入了如果输入 1 就恢复到最开始的视角这个功能，避免把视角移得太远。

### 3.3 结果图片

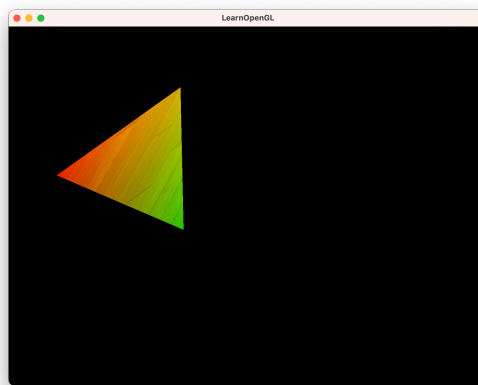


图 4: 实验 2 鼠标移动结果图 1

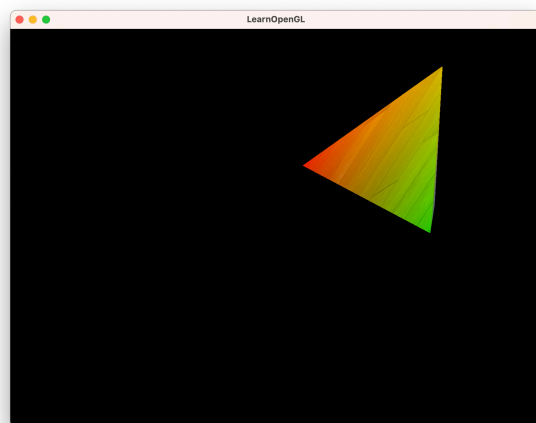


图 5: 实验 2 鼠标移动结果图 2

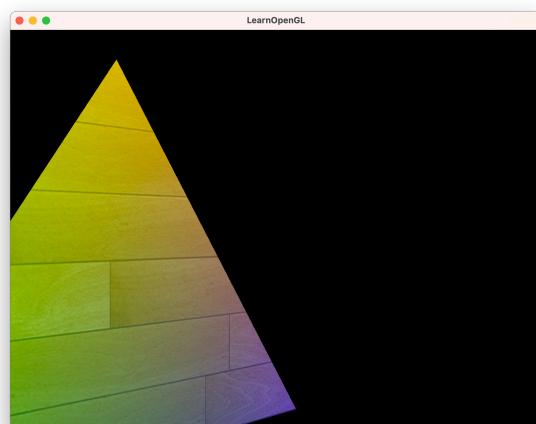


图 6: 实验 2 键盘 A+S 移动结果图

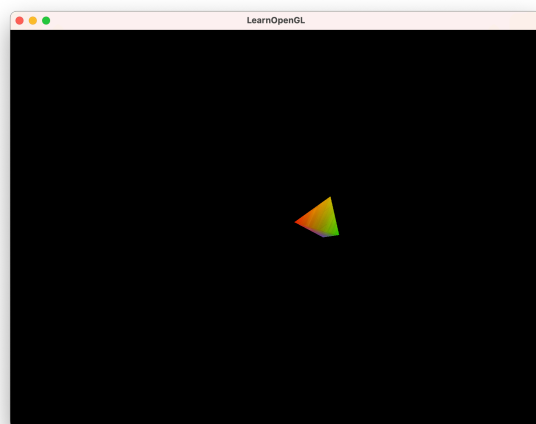


图 7: 实验 2 键盘 D+W 移动结果图

## 4 任务 3: 加入文字

### 4.1 任务要求

在任务 2 的基础上，在屏幕上显示当前摄像机位置、方向、坐标系矩阵的文本。

### 4.2 结果

经过长时间(从周四晚上到周五下午)探索,未能在 Mac 的 XCode 上链接 FreeType 库,总是报告一个奇怪的错误,上网搜索不能找到结果。故本题选择放弃。错误结果如图 8:

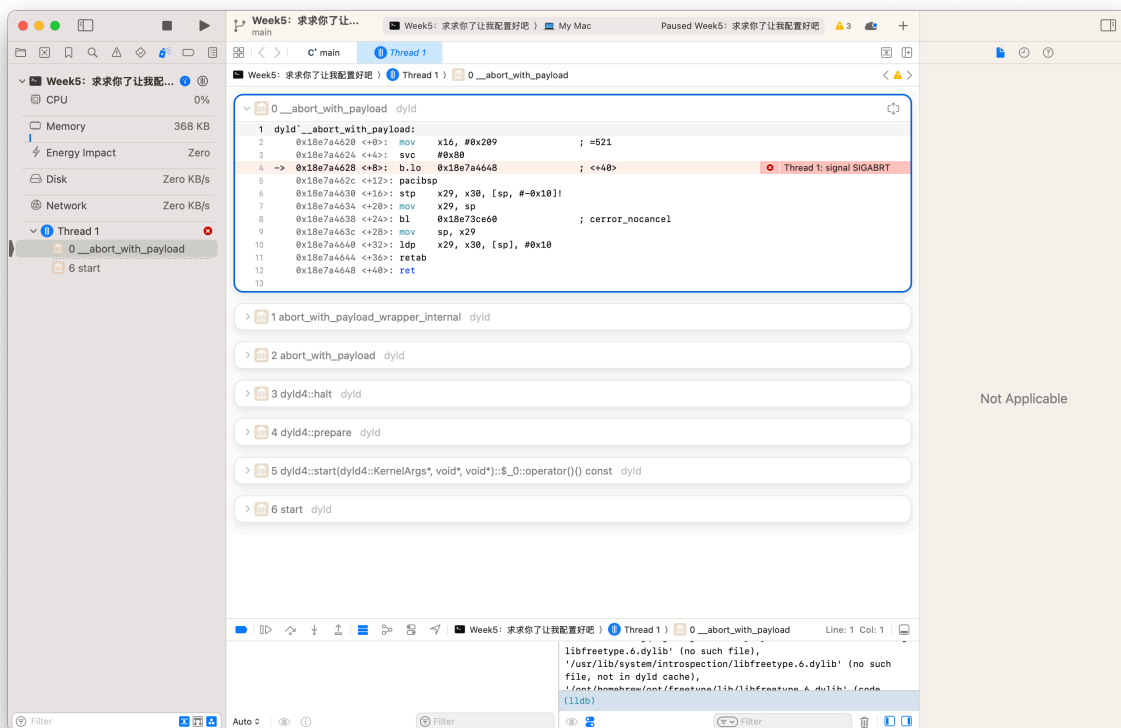


图 8: 实验 3 错误结果图